

ITA0605 - Machine Learning

List of Lab Exercises

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

Aim:

To implement and demonstrate the Candidate-Elimination Algorithm using a CSV file to find the set of all hypotheses consistent with the given training examples.

Algorithm:

1. Import the required libraries (`pandas`).
2. Read the training data from a CSV file.
3. Initialize the Specific hypothesis (S) with the first positive example.
4. Initialize the General hypothesis (G) with the most general hypothesis (?).
5. For each training example:
 - If it is positive, generalize the Specific hypothesis.
 - If it is negative, specialize the General hypothesis.
6. Remove inconsistent hypotheses.
7. Display the final Specific (S) and General (G) hypotheses.

Dataset 1:

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
sample_data = [
```

```
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
```

```
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
```

```
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
```

```
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Same', 'Yes'],
```

```
    ['Rainy', 'Warm', 'Normal', 'Weak', 'Warm', 'Same', 'No'],
```

```
    ['Sunny', 'Warm', 'Normal', 'Weak', 'Warm', 'Same', 'Yes'],
```

```
    ['Cloudy', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
```

```
    ['Sunny', 'Cold', 'High', 'Weak', 'Cool', 'Change', 'No']
```

```
]
```

```
column_names = [
```

```
    'Sky',
```

```
    'Temperature',
```

```
    'Humidity',
```

```
    'Wind',
```

```
    'Water',
```

```
    'Forecast',
```

```
    'PlayTennis'
```

```
]
```

```
data = pd.DataFrame(sample_data, columns=column_names)
```

```
print("Play Tennis Dataset:")
```

```
print(data)
```

```
concepts = np.array(data.iloc[:, :-1])
```

```
target = np.array(data.iloc[:, -1])
```

```
S = concepts[0].copy()
```

```
G = [["?" for i in range(len(S))] for j in range(len(S))]
```

```
for i, h in enumerate(concepts):
```

```
    if target[i] == "Yes":
```

```
        for x in range(len(S)):
```

```
if h[x] != S[x]:
```

```
    S[x] = "?"
```

```
    G[x][x] = "?"
```

```
else:
```

```
    for x in range(len(S)):
```

```
        if h[x] != S[x]:
```

```
            G[x][x] = S[x]
```

```
        else:
```

```
            G[x][x] = "?"
```

```
G = [
```

```
    g for g in G
```

```
    if g != ["?" for i in range(len(S))]
```

```
]
```

```
print("\n-----")
```

```
print("Candidate Elimination Results")
```

```
print("-----")
```

```
print("\nSpecific Hypothesis (S):")
```

```
print(S)
```

```
print("\nGeneral Hypothesis (G):")
```

for g in G:

print(g)

Output:

```
import pandas as pd
import numpy as np
sample_data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
column_names = ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast', 'EnjoySport']
data = pd.DataFrame(sample_data, columns=column_names)
concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])
S = concepts[0].copy()
G = [["?" for i in range(len(S))] for j in range(len(S))]
for i, h in enumerate(concepts):
    if target[i] == "Yes":
        for x in range(len(S)):
            if h[x] != S[x]:
                S[x] = "?"
                G[x][x] = "?"
    else:
        for x in range(len(S)):
            if h[x] != S[x]:
                G[x][x] = S[x]
            else:
                G[x][x] = "?"
G = [g for g in G if g != [ "?" for i in range(len(S))]]
print("Specific Hypothesis:")
print(S)
print("\nGeneral Hypothesis:")
for g in G:
    print(g)
```

```
... Specific Hypothesis:
['Sunny' 'Warm' '?' 'Strong' '?' '?']

General Hypothesis:
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
```

Dataset 2:

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
sample_data = [
    ['High','Good','Permanent','Yes','Young','Yes'],
    ['High','Good','Permanent','No','Middle','Yes'],
    ['Low','Poor','Temporary','No','Young','No'],
    ['Medium','Good','Permanent','Yes','Middle','Yes'],
    ['High','Average','Temporary','Yes','Old','No'],
    ['Medium','Good','Permanent','No','Young','Yes'],
    ['Low','Good','Permanent','Yes','Middle','Yes'],
    ['Low','Poor','Temporary','Yes','Old','No']
]
```

```
column_names = ['Income','CreditScore','Employment','Property','Age','LoanApproved']
```

```
data = pd.DataFrame(sample_data, columns=column_names)
```

```
concepts = np.array(data.iloc[:, :-1])
```

```
target = np.array(data.iloc[:, -1])
```

```
S = concepts[0].copy()
```

```
G = [["?" for i in range(len(S))] for j in range(len(S))]
```

```
for i, h in enumerate(concepts):
```

```
    if target[i] == "Yes":
```

```
        for x in range(len(S)):
```

```
            if h[x] != S[x]:
```

```
                S[x] = "?"
```

```
                G[x][x] = "?"
```

```
    else:
```

```
        for x in range(len(S)):
```

```
            if h[x] != S[x]:
```

```
                G[x][x] = S[x]
```

```
            else:
```

```
                G[x][x] = "?"
```

```
G = [g for g in G if g != ["?" for i in range(len(S))]]
```

```
print("Specific Hypothesis:")
```

```
print(S)
```

```
print("\nGeneral Hypothesis:")
```

```
for g in G:
```

```
    print(g)
```

Output:

```
import pandas as pd
import numpy as np

sample_data = [
    ['High', 'Good', 'Permanent', 'Yes', 'Young', 'Yes'],
    ['High', 'Good', 'Permanent', 'No', 'Middle', 'Yes'],
    ['Low', 'Poor', 'Temporary', 'No', 'Young', 'No'],
    ['Medium', 'Good', 'Permanent', 'Yes', 'Middle', 'Yes'],
    ['High', 'Average', 'Temporary', 'Yes', 'Old', 'No'],
    ['Medium', 'Good', 'Permanent', 'No', 'Young', 'Yes'],
    ['Low', 'Good', 'Permanent', 'Yes', 'Middle', 'Yes'],
    ['Low', 'Poor', 'Temporary', 'Yes', 'Old', 'No']
]

column_names = ['Income', 'CreditScore', 'Employment', 'Property', 'Age', 'LoanApproved']

data = pd.DataFrame(sample_data, columns=column_names)

concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])

S = concepts[0].copy()
G = [["?" for i in range(len(S))] for j in range(len(S))]

for i, h in enumerate(concepts):
    if target[i] == "Yes":
        for x in range(len(S)):
            if h[x] != S[x]:
                S[x] = "?"
                G[x][x] = "?"
```

```

        G[x][x] = "?"
    else:
        for x in range(len(S)):
            if h[x] != S[x]:
                G[x][x] = S[x]
            else:
                G[x][x] = "?"

G = [g for g in G if g != ["?" for i in range(len(S))]]

print("Specific Hypothesis:")
print(S)

print("\nGeneral Hypothesis:")
for g in G:
    print(g)

```

```

*** Specific Hypothesis:
['?' 'Good' 'Permanent' '?' '?']

General Hypothesis:
['?', 'Good', '?', '?', '?']
['?', '?', 'Permanent', '?', '?']

```

Dataset 3:

Program:

```

import pandas as pd
import numpy as np

sample_data = [
    ['High', 'Good', 'Yes', 'Good', 'High', 'Yes'],
    ['High', 'Excellent', 'Yes', 'Good', 'High', 'Yes'],
    ['Medium', 'Average', 'No', 'Average', 'Medium', 'No'],
    ['High', 'Good', 'Yes', 'Excellent', 'High', 'Yes'],
    ['Low', 'Poor', 'No', 'Average', 'Low', 'No'],
    ['High', 'Good', 'Yes', 'Good', 'Medium', 'Yes']
]

column_names = [
    'CGPA',
    'Communication',
    'Internship',
    'Programming',
    'Aptitude',
    'Placed'
]

```

```
]
data = pd.DataFrame(sample_data, columns=column_names)
```

```
print("Student Placement Dataset:")
print(data)
```

```
concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])
S = concepts[0].copy()
G = [["?" for i in range(len(S))] for j in range(len(S))]
```

```
for i, h in enumerate(concepts):
```

```
    if target[i] == "Yes":
```

```
        # Positive example
        for x in range(len(S)):
```

```
            if h[x] != S[x]:
                S[x] = "?"
                G[x][x] = "?"
```

```
    else:
```

```
        # Negative example
        for x in range(len(S)):
```

```
            if h[x] != S[x]:
                G[x][x] = S[x]
```

```
    else:
        G[x][x] = "?"
```

```
G = [
    g for g in G
    if g != ["?" for i in range(len(S))]
]
```

```
print("\n-----")
print("Candidate Elimination Results")
print("-----")
```



```
print("\nSpecific Hypothesis (S):")
print(S)
```

```
print("\nGeneral Hypothesis (G):")
for g in G:
    print(g)
```

Output:

```
*** Student Placement Dataset:
      CGPA Communication Internship Programming Aptitude Placed
0      High           Good         Yes         Good      High     Yes
1      High      Excellent         Yes         Good      High     Yes
2  Medium      Average          No      Average  Medium     No
3      High           Good         Yes      Excellent      High     Yes
4       Low           Poor          No      Average       Low     No
5      High           Good         Yes         Good  Medium     Yes

-----
Candidate Elimination Results
-----

Specific Hypothesis (S):
['High' '?' 'Yes' '?' '?']

General Hypothesis (G):
['High', '?', '?', '?', '?']
['?', '?', 'Yes', '?', '?']
```

Dataset 4:

Program:

```
import pandas as pd
import numpy as np

sample_data = [
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Positive'],
    ['Yes', 'Yes', 'No', 'Yes', 'Yes', 'Positive'],
    ['No', 'Yes', 'Yes', 'No', 'No', 'Negative'],
    ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'Positive'],
    ['No', 'No', 'Yes', 'Yes', 'No', 'Negative'],
    ['Yes', 'Yes', 'No', 'No', 'Yes', 'Positive'],
    ['Yes', 'No', 'Yes', 'Yes', 'Yes', 'Positive'],
    ['No', 'No', 'No', 'No', 'No', 'Negative']
]
```

```

column_names = [
    'Fever',
    'Cough',
    'Headache',
    'BodyPain',
    'Fatigue',
    'Disease'
]
data = pd.DataFrame(sample_data, columns=column_names)

```

```

print("Disease Diagnosis Dataset:")
print(data)

```

```

concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])
S = concepts[0].copy()
G = [["?" for i in range(len(S))] for j in range(len(S))]

```

```

for i, h in enumerate(concepts):

```

```

    if target[i] == "Positive":
        for x in range(len(S)):

```

```

            if h[x] != S[x]:
                S[x] = "?"
                G[x][x] = "?"

```

```

        else:
            for x in range(len(S)):

```

```

                if h[x] != S[x]:
                    G[x][x] = S[x]

```

```

                else:
                    G[x][x] = "?"

```

```

G = [
    g for g in G
    if g != ["?" for i in range(len(S))]
]

```

]

```
print("\n-----")
print("Candidate Elimination Results")
print("-----")
```

```
print("\nSpecific Hypothesis (S):")
print(S)
```

```
print("\nGeneral Hypothesis (G):")
for g in G:
    print(g)
```

Output:

```
*** Disease Diagnosis Dataset:
    Fever Cough Headache BodyPain Fatigue  Disease
0   Yes   Yes    Yes     Yes     Yes   Positive
1   Yes   Yes    No      Yes     Yes   Positive
2   No    Yes    Yes     No      No    Negative
3   Yes   Yes    Yes     No      Yes   Positive
4   No    No     Yes     Yes     No    Negative
5   Yes   Yes    No      No      Yes   Positive
6   Yes   No     Yes     Yes     Yes   Positive
7   No    No     No      No      No    Negative

-----
Candidate Elimination Results
-----

Specific Hypothesis (S):
['Yes' '?' '?' '?' 'Yes']

General Hypothesis (G):
['Yes', '?', '?', '?', '?']
['?', '?', '?', '?', 'Yes']
```

Dataset 5:

Program:

```
import pandas as pd
import numpy as np

sample_data = [
    ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes'],
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'No', 'No', 'No'],
    ['Yes', 'No', 'Yes', 'No', 'Yes', 'Yes'],
    ['No', 'Yes', 'No', 'Yes', 'No', 'No'],
    ['Yes', 'Yes', 'Yes', 'No', 'No', 'Yes'],
    ['Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'Yes', 'No', 'No', 'No']
]

column_names = [
    'ContainsLink',
    'ContainsOffer',
    'UnknownSender',
    'Attachment',
    'ManyRecipients',
    'Spam'
]

data = pd.DataFrame(sample_data, columns=column_names)

print("Email Spam Detection Dataset:")
print(data)

concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])
S = concepts[0].copy()
G = [["?" for i in range(len(S))] for j in range(len(S))]
for i, h in enumerate(concepts):

    if target[i] == "Yes":
        for x in range(len(S)):

            if h[x] != S[x]:
```

```

        S[x] = "?"
        G[x][x] = "?"

    else:
        for x in range(len(S)):

            if h[x] != S[x]:
                G[x][x] = S[x]

            else:
                G[x][x] = "?"

G = [
    g for g in G
    if g != ["?" for i in range(len(S))]
]

print("\n-----")
print("Candidate Elimination Results")
print("-----")

print("\nSpecific Hypothesis (S):")
print(S)

print("\nGeneral Hypothesis (G):")
for g in G:
    print(g)

```

Output:

```

... Email Spam Detection Dataset:
    ContainsLink ContainsOffer UnknownSender Attachment ManyRecipients Spam
0             Yes             Yes             Yes             No             Yes Yes
1             Yes             Yes             Yes             Yes             Yes Yes
2             No              No              No              No              No  No
3             Yes             No              Yes             No              Yes Yes
4             No              Yes             No              Yes             No  No
5             Yes             Yes             Yes             No              No  Yes
6             Yes             No              Yes             Yes             Yes Yes
7             No              No              Yes             No              No  No

-----
Candidate Elimination Results
-----

Specific Hypothesis (S):
['Yes' '?' 'Yes' '?' '?']

General Hypothesis (G):
['Yes', '?', '?', '?', '?']

```

Result:

The Candidate-Elimination algorithm was successfully implemented using a CSV file. It generated the Specific Hypothesis (S) and the General Hypothesis (G), representing the version space of all hypotheses that are consistent with the given training examples.

